

LAPORAN PRAKTIKUM

▪ Identitas Praktikum

Nama MK : Praktikum Pemrograman Berorientasi Objek
Kode MK : CCK2AAB4
Bobot SKS : 1 SKS
Tempat : L-MM, Gedung IOT, lantai 3
Hari, tanggal : Selasa, 2 Juni 2026
Jam : 09:30 - 12:30 WIB
Topik praktikum : Modul 15 - JDBC Part 3 & Session

▪ Identitas Mahasiswa

Nama lengkap : 'Aarif Rahmaan Jalaluddin Faqiih
NIM : 103112430182
Program Studi : S-1 Teknik Informatika

▪ Pendahuluan

Pada praktikum Modul 15, aplikasi web CRUD yang sudah dibuat pada modul sebelumnya ditambahkan fitur autentikasi menggunakan session. Session menyimpan username setelah proses login berhasil sehingga aplikasi dapat mengenali pengguna pada permintaan HTTP berikutnya. Login menerima username admin dan password admin123. Ketika logout dipilih, session dihapus dan halaman CRUD tidak dapat dibuka sebelum login kembali.

▪ form_login.jsp: Kode Program

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%
    if (session.getAttribute("username") != null) {
        response.sendRedirect(request.getContextPath() + "/viewusers.jsp");
        return;
    }
%>
<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <title>Login</title>
    </head>
    <body>
        <h1>Login</h1>

        <% if (request.getAttribute("message") != null) { %>
            <p><%= request.getAttribute("message") %></p>
        <% } %>

        <% if (request.getAttribute("loginError") != null) { %>
            <p><%= request.getAttribute("loginError") %></p>
        <% } %>

        <form method="post" action="<%= request.getContextPath() %>/LoginServlet">
            <table>
                <tr>
                    <td>Username:</td>
                    <td><input type="text" name="user" autocomplete="username"
required></td>
                </tr>
                <tr>
                    <td>Password:</td>
                    <td><input type="password" name="pass" autocomplete="current-password"
required></td>
                </tr>
                <tr>
                    <td colspan="2"><input type="submit" value="Login"></td>
                </tr>
            </table>
        </form>
    </body>
</html>
```

- **form_login.jsp: Penjelasan**

Code `<%@page contentType="text/html" pageEncoding="UTF-8"%>` berfungsi untuk menetapkan bahwa file JSP ini akan mengirim response dengan tipe text/html dan encoding UTF-8, sehingga karakter yang ditulis di dalam halaman dapat diproses dengan benar oleh browser. Directive ini dieksekusi ketika JSP dikompilasi menjadi servlet. Bagian `<% if (session.getAttribute("username") != null) { ... } %>` berfungsi untuk melakukan pengecekan session login. Jika atribut username sudah ada di dalam session, maka user dianggap sudah login, sehingga request langsung dialihkan ke halaman viewusers.jsp menggunakan `response.sendRedirect()`. Perintah `return` ditambahkan agar proses eksekusi halaman JSP berhenti setelah proses redirect dilakukan. Tag `<!DOCTYPE html>` sampai `<body>` merupakan struktur dasar dokumen HTML. Pada bagian `<h1>Login</h1>`, program menampilkan judul halaman login yang akan dilihat oleh user ketika halaman pertama kali dibuka. Blok `<% if (request.getAttribute("message") != null) { %>` berfungsi untuk mengecek apakah terdapat atribut message yang dikirim dari servlet atau halaman lain. Jika atribut tersebut ada, maka isi pesan akan dicetak menggunakan expression tag `<%= ... %>`. Cara kerja yang sama juga diterapkan pada atribut loginError, tetapi tujuannya untuk menampilkan informasi kesalahan ketika proses login gagal. Tag `<form method="post" action="<%= request.getContextPath() %>/LoginServlet">` berfungsi untuk mengirim data login ke LoginServlet menggunakan metode HTTP POST. Penggunaan `request.getContextPath()` bertujuan agar path aplikasi tetap sesuai walaupun nama project berubah atau dipindahkan ke server lain. Bagian `<input type="text" name="user">` dan `<input type="password" name="pass">` berfungsi untuk menerima input username dan password dari user. Attribute `autocomplete` digunakan agar browser dapat mengenali jenis data yang dimasukkan. Sementara itu, attribute `required` membuat form tidak dapat dikirim jika kolom masih kosong.

- **LoginServlet.java: Kode Program**

```
package com.javatpoint.servlet;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import java.io.IOException;

@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {

    private static final String ADMIN_USERNAME = "admin";
```

```

private static final String ADMIN_PASSWORD = "admin123";

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    HttpSession session = request.getSession(false);

    if (request.getParameter("logout") != null) {
        if (session != null) {
            session.invalidate();
        }
        request.setAttribute("message", "Logout berhasil.");
        request.getRequestDispatcher("/form_login.jsp").forward(request, response);
        return;
    }

    if (session != null && session.getAttribute("username") != null) {
        response.sendRedirect(request.getContextPath() + "/viewusers.jsp");
        return;
    }

    request.getRequestDispatcher("/form_login.jsp").forward(request, response);
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String username = request.getParameter("user");
    String password = request.getParameter("pass");

    if (ADMIN_USERNAME.equals(username) && ADMIN_PASSWORD.equals(password)) {
        HttpSession oldSession = request.getSession(false);
        if (oldSession != null) {
            oldSession.invalidate();
        }

        HttpSession session = request.getSession(true);
        session.setAttribute("username", username);
        response.sendRedirect(request.getContextPath() + "/viewusers.jsp");
        return;
    }

    request.setAttribute("loginError", "Username atau password salah.");
    request.getRequestDispatcher("/form_login.jsp").forward(request, response);
}
}

```

- **LoginServlet.java: Penjelasan**

Class LoginServlet merupakan turunan dari HttpServlet yang bertugas untuk menangani proses login, pengecekan session, dan logout pada aplikasi berbasis Jakarta Servlet. Anotasi @WebServlet("/LoginServlet") berfungsi untuk memberi tahu server bahwa request dengan URL /LoginServlet akan diarahkan ke class ini. Selain itu, terdapat dua konstanta yaitu ADMIN_USERNAME dan ADMIN_PASSWORD yang berisi username serta password admin yang digunakan sebagai data autentikasi. Method doGet() bertugas untuk menangani request dengan metode GET. Pada bagian HttpSession session = request.getSession(false), program mencoba mengambil session yang sudah ada tanpa membuat session baru. Setelah itu, program melakukan pengecekan parameter logout. Jika parameter tersebut ada, maka session lama akan dihapus menggunakan session.invalidate(), kemudian program mengirim pesan "Logout berhasil." ke form_login.jsp. Jika tidak ada proses logout dan session masih memiliki attribute username, maka user akan diarahkan ke halaman viewusers.jsp menggunakan response.sendRedirect(). Jika kedua kondisi tersebut tidak terpenuhi, maka halaman login form_login.jsp akan dibuka. Method doPost() bertugas untuk menangani proses login ketika form dikirim menggunakan metode POST. Pada bagian ini, program mengambil nilai username dan password dari form melalui request.getParameter("user") dan request.getParameter("pass"). Setelah itu dilakukan pengecekan apakah username dan password yang dimasukkan sama dengan nilai ADMIN_USERNAME dan ADMIN_PASSWORD. Jika hasil pengecekan sesuai, maka session lama akan dihapus terlebih dahulu guna mencegah penggunaan session sebelumnya. Setelah itu dibuat session baru menggunakan request.getSession(true), kemudian username disimpan ke dalam session melalui session.setAttribute("username", username). Selanjutnya user akan diarahkan ke halaman viewusers.jsp. Jika username atau password tidak sesuai, maka program tidak membuat session baru. Sebagai gantinya, program menyimpan pesan kesalahan "Username atau password salah." ke attribute loginError, kemudian request diteruskan kembali ke halaman form_login.jsp menggunakan RequestDispatcher. Dengan cara ini, halaman login dapat menampilkan informasi bahwa proses login gagal tanpa berpindah URL.

- **AuthFilter.java: Penyesuaian Project**

```
package com.javatpoint.filter;

import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
```

```

import jakarta.servlet.annotation.WebFilter;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import java.io.IOException;

@WebFilter(urlPatterns = {
    "/viewusers.jsp",
    "/adduserform.jsp",
    "/adduser.jsp",
    "/adduser-success.jsp",
    "/adduser-error.jsp",
    "/userform.jsp",
    "/editform.jsp",
    "/edituser.jsp",
    "/deleteuser.jsp"
})
public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain
chain)
        throws IOException, ServletException {
        HttpServletRequest httpRequest = (HttpServletRequest) request;
        HttpServletResponse httpResponse = (HttpServletResponse) response;
        HttpSession session = httpRequest.getSession(false);

        if (session != null && session.getAttribute("username") != null) {
            chain.doFilter(request, response);
            return;
        }

        httpResponse.sendRedirect(httpRequest.getContextPath() + "/LoginServlet");
    }
}

```

▪ **AuthFilter.java: Penjelasan**

Code AuthFilter merupakan class yang mengimplementasikan interface Filter, bertujuan untuk melakukan pengecekan login sebelum pengguna dapat mengakses halaman tertentu. Maka dari itu, setiap request yang menuju file seperti viewusers.jsp, adduserform.jsp, edituser.jsp, dan file lain yang tercantum pada @WebFilter akan melewati class ini terlebih dahulu sebelum diteruskan ke halaman tujuan. Code @WebFilter(urlPatterns = {...}) berfungsi untuk memberi tahu server bahwa class AuthFilter akan dijalankan ketika ada request menuju daftar URL yang sudah

ditentukan. Jika diterjemahkan secara fungsi, maka daftar URL tersebut merupakan halaman yang dilindungi sehingga tidak dapat diakses secara langsung tanpa melalui proses pengecekan login. Method `doFilter(ServletRequest request, ServletResponse response, FilterChain chain)` berfungsi untuk menerima request yang masuk, kemudian melakukan proses pengecekan session. Pada bagian `HttpServletRequest httpRequest = (HttpServletRequest) request` dan `HttpServletResponse httpResponse = (HttpServletResponse) response`, terjadi proses konversi object umum servlet menjadi object HTTP agar method khusus HTTP seperti `getSession()` dan `sendRedirect()` dapat digunakan. Code `HttpSession session = httpRequest.getSession(false)` berfungsi untuk mengambil session yang sudah ada. Penggunaan parameter `false` berarti server tidak akan membuat session baru jika session sebelumnya tidak ditemukan. Maka dari itu, proses pengecekan login hanya dilakukan terhadap session yang memang sudah tersimpan sebelumnya. Bagian `if (session != null && session.getAttribute("username") != null)` berfungsi untuk memeriksa apakah session tersedia dan apakah attribute username sudah tersimpan. Jika kondisi tersebut terpenuhi, maka `chain.doFilter(request, response)` akan dijalankan sehingga request diteruskan ke halaman tujuan yang diminta pengguna. Jika session tidak ditemukan atau attribute username tidak ada, maka code `httpResponse.sendRedirect(httpRequest.getContextPath() + "/LoginServlet")` akan dijalankan. Perintah ini berfungsi untuk mengarahkan pengguna kembali ke halaman login, karena sistem menganggap pengguna belum melakukan proses autentikasi.

- **Perubahan Kode Program index.jsp**

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%
    if (session.getAttribute("username") == null) {
        response.sendRedirect(request.getContextPath() + "/LoginServlet");
    } else {
        response.sendRedirect(request.getContextPath() + "/viewusers.jsp");
    }
%>
```

- **Penjelasan index.jsp**

Code `if (session.getAttribute("username") == null)` berfungsi untuk melakukan pengecekan apakah session bernama username belum memiliki nilai. Jika kondisi ini terpenuhi, maka `response.sendRedirect(request.getContextPath() + "/LoginServlet")` akan mengarahkan user ke halaman login karena sistem menganggap user belum melakukan login. Tapi jika session username sudah memiliki nilai, maka `response.sendRedirect(request.getContextPath() + "/viewusers.jsp")` akan

mengarahkan user ke halaman viewusers.jsp, sehingga halaman tersebut hanya dapat diakses setelah proses login berhasil.

- **Perubahan Kode Program viewusers.jsp**

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<%@page import="com.javatpoint.dao.UserDao"%>
<%@page import="com.javatpoint.bean.User"%>
<%@page import="java.util.*"%>

<%@ taglib prefix="c" uri="jakarta.tags.core" %>

<!DOCTYPE html>

<html>
<head>
    <meta charset="UTF-8">
    <title>View Users</title>
</head>

<body>

    <h1>Users List</h1>

    <p>
        Login sebagai: ${sessionScope.username} |
        <a href="${pageContext.request.contextPath}/LoginServlet?logout=true">Logout</a>
    </p>

    <%
        List<User> list = UserDao.getAllRecords();
        request.setAttribute("list", list);
    %>

    <table border="1" width="90%">

        <tr>
            <th>Id</th>
            <th>Name</th>
            <th>Password</th>
            <th>Email</th>
            <th>Sex</th>
            <th>Country</th>
            <th>Edit</th>
            <th>Delete</th>
        </tr>
```



```

<c:forEach items="${list}" var="u">

    <tr>

        <td>${u.id}</td>
        <td>${u.name}</td>
        <td>${u.password}</td>
        <td>${u.email}</td>
        <td>${u.sex}</td>
        <td>${u.country}</td>

        <td>
            <a href="editform.jsp?id=${u.id}">
                Edit
            </a>
        </td>

        <td>
            <a href="deleteuser.jsp?id=${u.id}">
                Delete
            </a>
        </td>

    </tr>

</c:forEach>

</table>

<br>

<a href="adduserform.jsp">
    Add New User
</a>

</body>
</html>

```

▪ **Penjelasan viewusers.jsp**

Code `<%@page contentType="text/html" pageEncoding="UTF-8"%>` berfungsi untuk menentukan bahwa halaman JSP ini akan menghasilkan output HTML dengan encoding UTF-8. Setelah itu, beberapa baris `<%@page import %>` digunakan untuk mengimpor class `UserDao`, `User`, dan `java.util.*` agar class tersebut dapat diakses di dalam file JSP. Sementara itu, baris `<%@ taglib prefix="c" uri="jakarta.tags.core" %>` berfungsi untuk mengaktifkan JSTL Core sehingga tag seperti `<c:forEach>` dapat digunakan

untuk melakukan perulangan data. Pada bagian <body>, terdapat heading "Users List" dan informasi login yang diambil dari session melalui `${sessionScope.username}`. Artinya, username yang tersimpan di session akan ditampilkan bersama link logout yang mengarah ke `LoginServlet` dengan parameter `logout=true`. Maka dari itu, ketika user memilih logout, proses penghapusan session akan diproses oleh servlet tersebut. Bagian scriptlet JSP `<% ... %>` digunakan untuk mengambil seluruh data user dari database melalui `UserDao.getAllRecords()`. Hasil pengambilan data tersebut disimpan ke dalam objek `List<User>` yang bernama `list`, kemudian data tersebut dimasukkan ke request attribute menggunakan `request.setAttribute("list", list)`. Untuk itu, data yang sudah tersimpan di request dapat diakses kembali oleh JSTL pada bagian berikutnya. Tag `<table>` digunakan untuk membuat tabel data user. Baris `<tr>` pertama berisi header kolom seperti ID, Name, Password, Email, Sex, Country, Edit, dan Delete. Setelah itu, tag `<c:forEach items="${list}" var="u">` akan melakukan perulangan terhadap seluruh data user yang terdapat di dalam attribute `list`. Setiap objek user akan disimpan sementara ke variabel `u`, sehingga data seperti `${u.id}`, `${u.name}`, `${u.email}`, dan atribut lainnya dapat ditampilkan pada setiap baris tabel. Pada kolom Edit terdapat link `editform.jsp?id=${u.id}` yang bertujuan untuk mengirim ID user ke halaman edit. Dengan cara yang sama, kolom Delete memiliki link `deleteuser.jsp?id=${u.id}` guna mengirim ID user yang akan dihapus. Jadi, setiap baris data memiliki aksi edit dan hapus berdasarkan ID masing-masing user. Pada bagian paling bawah terdapat link `adduserform.jsp` yang berfungsi untuk membuka halaman penambahan user baru. Secara keseluruhan, file JSP ini bertugas sebagai halaman view yang mengambil data dari `UserDao`, menyimpannya ke request attribute, lalu menampilkan seluruh data user dalam bentuk tabel HTML menggunakan JSTL.

- **Perubahan Kode Program `userform.jsp`**

```
<a href="viewusers.jsp">View All Records</a> |  
<a href="<%= request.getContextPath() %>/LoginServlet?logout=true">Logout</a><br/>  
  
<h1>Add New User</h1>  
<form action="adduser.jsp" method="post">  
  <table>  
    <tr><td>Name:</td><td><input type="text" name="name"/></td></tr>  
    <tr><td>Password:</td><td>  
      <input type="password" name="password"/></td></tr>  
    <tr><td>Email:</td><td><input type="email" name="email"/></td></tr>  
    <tr><td>Sex:</td><td>  
      <input type="radio" name="sex" value="male"/>Male  
      <input type="radio" name="sex" value="female"/>Female </td></tr>  
    <tr><td>Country:</td><td>  
      <select name="country" style="width:155px">
```

```

        <option>Indonesia</option>
        <option>Korea</option>
        <option>Japan</option>
        <option>Other</option>
    </select>
</td></tr>
<tr><td colspan="2"><input type="submit" value="Add User"/></td></tr>
</table>
</form>

```

▪ **Penjelasan userform.jsp**

Tag `<a href="viewusers.jsp">View All Records</a>` berfungsi untuk mengarahkan pengguna ke halaman `viewusers.jsp` yang berisi daftar seluruh data user. Kemudian, tag `<a href="<%= request.getContextPath() %>/LoginServlet?logout=true">Logout</a>` berfungsi untuk menjalankan proses logout melalui `LoginServlet` dengan parameter `logout=true`. Penggunaan `request.getContextPath()` bertujuan agar path aplikasi tetap sesuai walaupun lokasi deployment berubah. Tag `<h1>Add New User</h1>` berfungsi untuk memberikan judul halaman. Setelah itu, tag `<form action="adduser.jsp" method="post">` berfungsi untuk mengirim data form ke file `adduser.jsp` menggunakan metode POST, karena data yang dikirim berasal dari input pengguna dan akan diproses oleh server. Tag `<table>` digunakan guna mengatur posisi setiap komponen input. Pada bagian `<input type="text" name="name"/>`, field tersebut berfungsi untuk menerima nama user. Kemudian `<input type="password" name="password"/>` berfungsi untuk menerima password, di mana karakter yang diketik tidak ditampilkan secara langsung. Selanjutnya, `<input type="email" name="email"/>` berfungsi untuk menerima alamat email. Bagian `<input type="radio" name="sex" value="male"/>` dan `<input type="radio" name="sex" value="female"/>` berfungsi untuk memberikan pilihan jenis kelamin. Karena kedua radio button mempunyai atribut `name` yang sama, maka pengguna hanya dapat memilih satu pilihan saja. Tag `<select name="country">` berfungsi untuk membuat daftar pilihan negara. Opsi yang tersedia terdiri dari Indonesia, Korea, Japan, dan Other. Ketika salah satu pilihan dipilih, nilai tersebut akan ikut dikirim bersama data form lainnya ke `adduser.jsp`. Bagian `<input type="submit" value="Add User"/>` berfungsi untuk mengirim seluruh data yang sudah diisi. Jadi, alur dari code ini adalah pengguna mengisi data user, kemudian data tersebut dikirim ke `adduser.jsp` untuk diproses lebih lanjut, misalnya disimpan ke database.

▪ **Perubahan Kode Program editform.jsp**

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
```

```

<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
        <title>Edit Form</title>
    </head>
    <body>
        <%@page import="com.javatpoint.dao.UserDao,com.javatpoint.bean.User"%>

        <%
            String id = request.getParameter("id");
            User u = UserDao.getRecordById(Integer.parseInt(id));
        %>

        <h1>Edit Form</h1>
        <a href="viewusers.jsp">View All Records</a> |
        <a href="<%= request.getContextPath() %>/LoginServlet?logout=true">Logout</a>
        <form action="edituser.jsp" method="post">
            <input type="hidden" name="id" value="<%=u.getId()%>" />
            <table>
                <tr><td>Name:</td><td>
                    <input type="text" name="name" value="<%=
u.getName()%>" /></td></tr>
                <tr><td>Password:</td><td>
                    <input type="password" name="password" value="<%=
u.getPassword()%>" /></td></tr>
                <tr><td>Email:</td><td>
                    <input type="email" name="email" value="<%=
u.getEmail()%>" /></td></tr>
                <tr><td>Sex:</td><td>
                    <input type="radio" name="sex" value="male" />Male
                    <input type="radio" name="sex" value="female" />Female </td></tr>
                <tr><td>Country:</td><td>
                    <select name="country">
                        <option>Indonesia</option>
                        <option>Korea</option>
                        <option>Japan</option>
                        <option>Other</option>
                    </select>
                </td></tr>
                <tr><td colspan="2"><input type="submit" value="Edit User" /></td></tr>
            </table>
        </form>

    </body>
</html>

```

- **Penjelasan editform.jsp**

Code `<%@page contentType="text/html" pageEncoding="UTF-8"%>` berfungsi untuk menentukan tipe konten yang akan dikirim oleh JSP serta encoding karakter yang digunakan. Pada bagian `<head>`, tag `<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">` berfungsi untuk memberi informasi encoding HTML yang digunakan oleh browser ketika halaman dijalankan. Selain itu, tag `<title>Edit Form</title>` berfungsi untuk memberikan judul halaman yaitu "Edit Form".

Code

`<%@page`

`import="com.javatpoint.dao.UserDao,com.javatpoint.bean.User"%>` berfungsi untuk memasukkan class UserDao dan User ke dalam file JSP ini. Setelah itu, bagian `<% ... %>` berfungsi untuk mengambil parameter id dari URL menggunakan `request.getParameter("id")`, kemudian nilai tersebut diubah menjadi tipe integer menggunakan `Integer.parseInt(id)`, lalu data user diambil dari database melalui `UserDao.getRecordById()`. Maka dari itu, seluruh informasi user yang akan diedit sudah tersedia di dalam object u. Tag `<a href="viewusers.jsp">View All Records</a>` berfungsi untuk mengarahkan pengguna ke halaman daftar seluruh data user. Sementara itu, code `<a href="<%= request.getContextPath() %>/LoginServlet?logout=true">Logout</a>` berfungsi untuk menjalankan proses logout melalui LoginServlet dengan parameter `logout=true`. Untuk itu, pengguna dapat keluar dari session aplikasi tanpa perlu mengetik URL secara manual. Tag `<form action="edituser.jsp" method="post">` berfungsi untuk mengirim data hasil edit ke file `edituser.jsp` menggunakan metode POST. Input hidden `<input type="hidden" name="id" value="<%=u.getId()%>" />` berfungsi untuk menyimpan ID user yang sedang diedit agar proses update mengetahui data mana yang harus diubah di database. Karena itu, walaupun ID tidak terlihat oleh pengguna, nilainya tetap ikut terkirim ketika form disubmit. Bagian `<input type="text" name="name" value="<%=u.getName()%>" />`, `<input type="password" name="password" value="<%=u.getPassword()%>" />`, dan `<input type="email" name="email" value="<%=u.getEmail()%>" />` berfungsi untuk menampilkan data user yang sebelumnya sudah tersimpan di database. Jadi, pengguna tidak perlu mengisi ulang seluruh data dari awal karena nilai lama sudah otomatis dimasukkan ke dalam form edit. Bagian radio button sex berfungsi untuk memilih jenis kelamin user, sedangkan tag `<select name="country">` berfungsi untuk memilih negara user dari beberapa pilihan yang sudah disediakan seperti Indonesia, Korea, Jepang, dan Other. Pada bagian akhir, tombol `<input type="submit" value="Edit User" />` berfungsi untuk mengirim seluruh perubahan data ke halaman `edituser.jsp` agar proses update data dapat dilakukan.

- **Demo Project**

- **Build dan Compile**

```
D:\crudweb\crudweb>mvn clean package
[INFO] Scanning for projects...
[INFO] -----< com.mycompany:crudweb >-----
[INFO] Building crudweb 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ war ]-----
[INFO] --- clean:3.2.0:clean (default-clean) @ crudweb ---
[INFO] Deleting D:\crudweb\crudweb\target
[INFO] --- resources:3.4.0:resources (default-resources) @ crudweb ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] --- compiler:3.8.1:compile (default-compile) @ crudweb ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 4 source files to D:\crudweb\crudweb\target\classes
[INFO] --- resources:3.4.0:testResources (default-testResources) @ crudweb ---
[INFO] skip non existing resourceDirectory D:\crudweb\crudweb\src\test\resources
[INFO] --- compiler:3.8.1:testCompile (default-testCompile) @ crudweb ---
[INFO] No sources to compile
[INFO] --- surefire:3.5.4:test (default-test) @ crudweb ---
[INFO] No tests to run.
[INFO] --- war:3.3.2:war (default-war) @ crudweb ---
[INFO] Packaging webapp
[INFO] Assembling webapp [crudweb] in [D:\crudweb\crudweb\target\crudweb-1.0-SNAPSHOT]
[INFO] Processing war project
[INFO] Copying webapp resources [D:\crudweb\crudweb\src\main\webapp]
[INFO] Building war: D:\crudweb\crudweb\target\crudweb-1.0-SNAPSHOT.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 5.802 s
[INFO] Finished at: 2026-06-27T13:19:17+07:00
[INFO] -----
```

- **Isi username admin dan password yang salah**

← → ↻ localhost:8080/crudweb/LoginServlet

Login

Username atau password salah.

Username:

Password:

Login

'Aarif Rahman Jalaluddin Faqih
103112430182
S1IF-12-06

- **Isi username admin dan password admin123**

← → ↻ localhost:8080/crudweb/viewusers.jsp

Users List

Login sebagai: admin | [Logout](#)

Id	Name	Password	Email	Sex	Country	Edit	Delete
--------------------	----------------------	--------------------------	-----------------------	---------------------	-------------------------	----------------------	------------------------

[Add New User](#)



- **Logout**

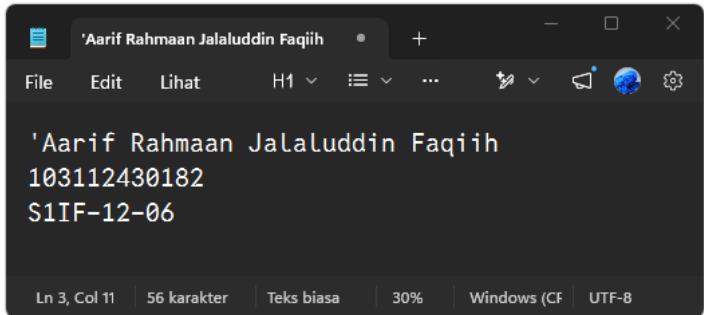
← → ↻ localhost:8080/crudweb/LoginServlet?logout=true

Login

Logout berhasil.

Username:

Password:



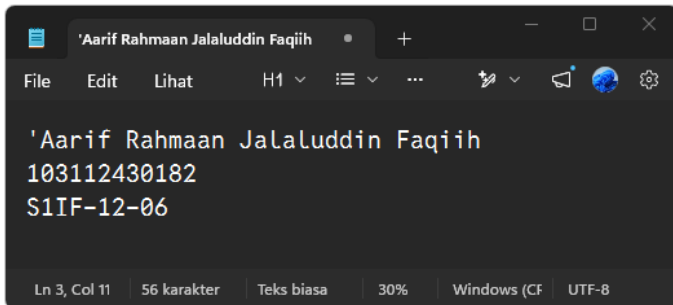
- **Mencoba akses ke halaman viewusers.jsp setelah logout**

← → ↻ http://localhost:8080/crudweb/viewusers.jsp

Login

Username:

Password:



- **Kesimpulan**
- **Link GitHub**